

Agent 工程：一门新学科

Agent Engineering: A New Discipline

LangChain · The LangChain Team · 2025-12-09 · 原文 / Source

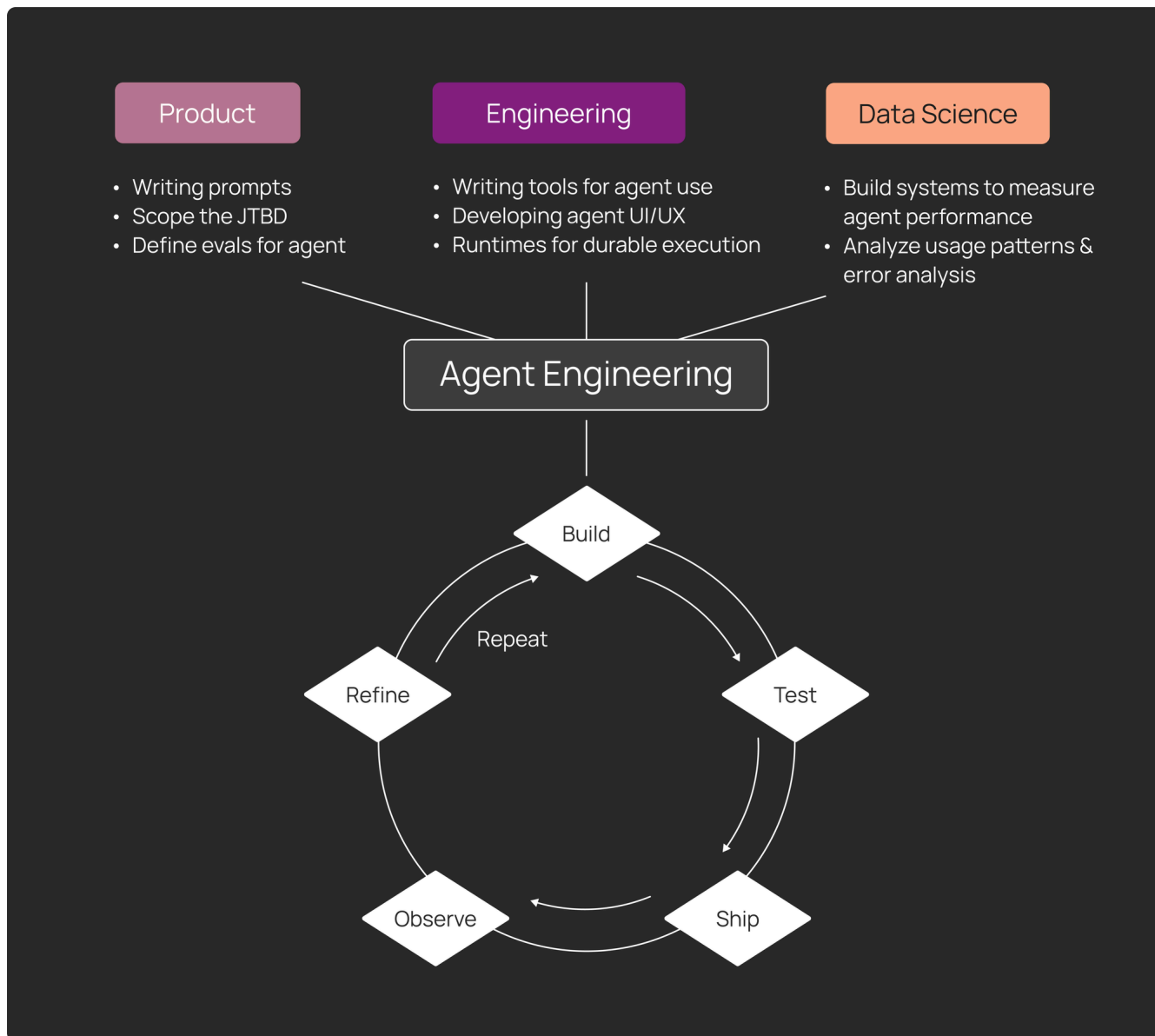
完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网站导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

如果你构建过 Agent，就会知道，“在我的机器上能用”与“在生产环境中能用”之间，可能存在巨大的差距。传统软件假设你大体了解输入，并且能够定义输出。Agent 却两者都无法保证：用户真的可以说任何话，而可能出现的行为几乎没有边界。这正是它们强大的原因，也正是它们可能以你未曾预料的方式偏离正轨的原因。

过去 3 年，我们见过数千个团队努力应对这一现实。那些成功将可靠产品交付到生产环境的团队——例如 Clay、Vanta、LinkedIn 和 Cloudflare——并没有照搬传统软件开发的方法。它们正在开创一套新实践：**Agent 工程**。

什么是 Agent 工程？

Agent 工程，是通过迭代改进，将非确定性的 LLM 系统打造成可靠生产体验的过程。这是一个循环过程：构建、测试、发布、观察、改进，再次循环。



关键在于，发布并不是最终目标。它只是让你持续获得新认识、改进 Agent 的方式。要做出有意义的改进，就需要理解生产环境中正在发生什么。完成这个循环的速度越快，Agent 就越可靠。

我们把 Agent 工程视为一门新学科，它将 3 类技能结合起来，让它们协同发挥作用：

- **产品思维**定义任务范围，塑造 Agent 行为。这包括：
 - 编写驱动 Agent 行为的提示词（通常有数百行，甚至数千行）。良好的沟通和写作能力在这里至关重要。
 - 深入理解 Agent 所复现的“待完成的工作”。
 - 定义评测，检验 Agent 的表现是否符合这项“待完成的工作”的要求。

- **工程能力**构建让 Agent 能够投入生产的基础设施。这包括：
 - 编写供 Agent 使用的工具。
 - 开发 Agent 交互的 UI/UX，包括流式输出、中断处理等。
 - 构建稳健的运行时，处理持久执行、需要人类介入时的暂停，以及记忆管理。
- **数据科学**持续衡量并改进 Agent 的表现。这包括：
 - 构建评测、A/B 测试、监控等系统，衡量 Agent 的表现与可靠性。
 - 分析使用模式并开展错误分析，因为相较于传统软件，用户使用 Agent 的方式更加广泛。

Agent 工程体现在哪里

Agent 工程并不是一种新的岗位名称。它是一组职责：当现有团队开始构建能够推理、适应变化且行为难以预测的系统时，就需要承担这些职责。如今能交付可靠 Agent 的组织，正在拓展工程、产品与数据团队的技能，以满足非确定性系统的要求。

这套实践通常体现在以下工作中：

- **软件工程师和机器学习工程师**编写提示词、构建供 Agent 使用的工具，追踪 Agent 为什么发起特定的工具调用，并改进底层模型。
- **平台工程师**构建 Agent 基础设施，处理持久执行与人类参与的工作流。
- **产品经理**编写提示词、定义 Agent 的任务范围，并确保 Agent 解决的是正确的问题。
- **数据科学家**衡量 Agent 的可靠性，识别改进机会。

这些团队拥抱快速迭代。你经常会看到，软件工程师追踪错误后，把发现交给产品经理据此调整提示词；也可能是产品经理发现任务范围存在问题，需要工程师提供新工具。大家都认识到：让 Agent 变得稳健，真正依靠的是这样的循环——观察生产环境中的行为，再根据所学到的东西系统地改进。

为什么需要 Agent 工程？为什么是现在？

两个根本性的变化，使 Agent 工程成为必要。

首先，LLM 已经足够强大，能够处理复杂的多步骤工作流。我们已经看到，Agent 开始承担整项工作，而不只是单个任务。Clay 用 Agent 处理从潜在客户研究到个性化联系、再到 CRM 更新的全部流程。LinkedIn 用 Agent 为招聘筛查海量人才库，对候选人排序，并即时呈现最合适的人选。我们正在跨越一个门槛：Agent 开始在生产环境中交付实质性的商业价值。

其次，这种能力伴随着真实存在的不可预测性。简单的 LLM 应用虽然也具有非确定性，但其行为范围通常更受限制。Agent 则不同。它们会进行多步推理、调用工具，并根据上下文调整行为。让 Agent 变得有用的那些特性，也使其行为有别于传统软件。这通常意味着：

- **每一种输入都可能是边缘情况。**当用户可以用自然语言询问任何事情时，就不存在所谓“正常”的输入。当你输入“让它更出彩”，或者“按上次那样做，但换一种方式”时，Agent 和人一样，可能对提示词作出不同解读。
- **你不能再用老办法调试。**由于大量逻辑存在于模型内部，你必须检查每一次决策与工具调用。提示词或配置中的细微调整，都可能造成行为上的巨大变化。

- **“能用”并不是一个非黑即白的判断。** Agent 的可用率即使达到 99.99%，仍可能偏离正轨、无法正常完成工作。对于真正重要的问题，答案未必只是简单的是或否：Agent 作出的决策是否正确？使用工具的方式是否恰当？是否遵循了指令背后的意图？

把这一切放在一起看：Agent 正在运行真实且影响重大的工作流，却又展现出传统软件方法难以应对的行为。这为一门新学科创造了机会，也产生了需求。Agent 工程让你能够发挥 LLM 的能力，同时构建在生产环境中真正值得信赖的系统。

Agent 工程在实践中是什么样的？

Agent 工程遵循的原则与传统软件开发不同。要获得可靠的 Agent 系统，发布本身就是学习的方式，而不是学完以后才做的事。

我们看到，成功的工程团队通常按类似下面的节奏开发 Agent：

- **构建 Agent 的基础。**从设计 Agent 的基础结构开始，无论它是一次带工具的简单 LLM 调用，还是复杂的多 Agent 系统。架构取决于你需要多少工作流（确定性的逐步流程），以及多少自主性（由 LLM 驱动的决策）。
- **针对你能想到的场景进行测试。**用示例场景测试 Agent，发现提示词、工具定义和工作流中的明显问题。传统软件可以预先规划用户流程，而对于自然语言输入，你无法预见用户交互的每一种方式。因此，要将思路从“穷尽测试，然后发布”转变为“合理测试，通过发布弄清什么真正重要”。
- **通过发布观察真实世界的行为。**一旦发布，你就会立刻看到未曾考虑过的输入；每一条生产环境执行轨迹，都展示了 Agent 实际需要处理什么。
- **观察。**追踪每一次交互，查看完整对话、每一个被调用的工具，以及 Agent 每次决策所依据的确切上下文。针对生产数据运行评测，衡量 Agent 的质量，无论你关注的是准确性、时延、用户满意度，还是其他标准。
- **改进。**识别出失败模式后，通过编辑提示词、修改工具定义来改进。这个过程始终持续进行：你可以把有问题的案例重新加入示例场景集，用于回归测试。
- **重复。**发布改进，观察生产环境发生了什么变化。每一次循环都会让你对用户如何与 Agent 交互，以及在你的场景中可靠性究竟意味着什么，获得新的认识。

工程实践的新标准

如今能交付可靠 Agent 的团队有一个共同点：它们不再试图在发布前就让 Agent 完美无缺，而是开始把生产环境当作最主要的老师。换句话说，就是追踪每一次决策、大规模开展评测，并以天为单位发布改进，而不是按季度推进。

Agent 工程正在兴起，是因为新的机会要求我们这样做。Agent 已经能够处理过去需要人类判断的工作流，但前提是你能够让它足够可靠、值得信赖。这里没有捷径，只有系统性的迭代工作。问题不在于 Agent 工程会不会成为标准实践，而在于你的团队能多快采用它，从而释放 Agent 的能力。